

REQUIREMENT ANALYSIS PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

1. Project Overview

ComicCraft is a web-based application designed to generate personalized comic book stories and illustrations from user-provided prompts. The system uses FastAPI, Google Gemini models, and Stable Diffusion to automate story outline generation, narration, dialogue, illustration creation, comic preview, and PDF export.

2. Purpose of Requirement Analysis

The requirement analysis identifies what ComicCraft must accept, process, generate, and display. It defines the functional and technical requirements needed to provide a complete flow from user input to a downloadable comic.

3. Stakeholders / Intended Users

- Storytellers and casual users who want to create comics without requiring drawing skills.
- Students and beginners interested in AI-assisted storytelling.
- Artists and creators who want assistance with story and illustration generation.
- Developers and testers who interact with the FastAPI API endpoints.

4. Functional Requirements

4.1 User Input Requirements

- The system shall provide a web form for entering a Story Prompt.
- The system shall accept a Main Character Name.
- The system shall accept a Setting such as forest, school, city, or space.
- The system shall accept a Story Tone such as dramatic, funny, poetic, or light-hearted.
- The system shall accept an Art Style such as anime, pixel art, comic book, or realistic.
- The form shall submit the information to the FastAPI backend using a POST request.

4.2 Story Generation Requirements

- The system shall generate a structured 5-panel comic outline.
- Each panel shall contain a panel number, title, scene description, and image-generation prompt.
- Gemini Flash shall be used for fast and structured comic panel outline generation.
- Gemini Pro shall expand the outline into detailed narration and character dialogue.

4.3 Image Generation Requirements

- The system shall generate an illustration for each comic panel.
- Stable Diffusion shall generate comic-style images from image prompts.
- Generated images shall be associated with their corresponding panels.
- Generated image files shall be stored for use in the comic preview and PDF export.

4.4 Comic Layout Requirements

- The system shall match each generated image with its panel story.
- The layout shall contain panel number, image path, and text.
- The comic preview shall display the panels sequentially.
- Each panel shall show its title, illustration, scene description, captions, and narration.

4.5 PDF Export Requirements

- The system shall provide a download option for the completed comic.
- The system shall compile panel images and narrative text into a multi-page PDF.
- The PDF shall be generated using FPDF.
- The exported PDF shall be saved in the designated exports location.
- After export, the user shall receive a success confirmation.

4.6 API Requirements

- The / route shall load the homepage.
- The /generate route shall process form-based comic generation.
- The /generate-comic/json route shall accept JSON input and return layout data and the PDF path.
- The /test-image route shall support direct image-generation testing.
- The /export-success route shall display the export confirmation page.
- FastAPI shall use form parameters and Pydantic validation for appropriate input handling.

5. Non-Functional Requirements

5.1 Usability

- The interface should be simple and user-friendly.
- The navigation should maintain a direct flow from input to comic generation.
- The comic preview should be visually clear and organized panel-by-panel.

5.2 Performance

- Gemini Flash is selected for fast structured outline generation.
- The application should efficiently coordinate text generation, image generation, layout building, and PDF export.
- The application should support local development and testing using Uvicorn.

5.3 Maintainability

- The application should use modular components such as `gemini_flash.py`, `gemini_pro.py`, `image_generator.py`, `layout_builder.py`, `exporters.py`, and `routes.py`.
- Frontend templates should remain separated by functionality.
- The architecture should allow future enhancements such as user profiles, comic libraries, and multi-page story arcs.

5.4 Security / Configuration

- Sensitive Gemini API keys and Hugging Face API tokens shall be stored using environment variables or a `.env` file.

- Project-specific Python dependencies shall be managed in a virtual environment.
- API errors and unexpected failures shall be handled through FastAPI HTTPException and try-except mechanisms.

6. Hardware and Software Requirements

- Python and pip.
- FastAPI framework.
- Uvicorn ASGI server.
- HTML, CSS, and Jinja2.
- Google Gemini API.
- Hugging Face Diffusers and Stable Diffusion.
- PyTorch basics and required model dependencies.
- FPDF for PDF generation.
- Pillow for image handling.
- Git for version control.
- A local web browser for accessing the application.

7. Input–Process–Output Requirements

Input:

- Story prompt, character name, setting, story tone, and art style.

Process:

- Validate and receive user input.
- Generate a 5-panel outline using Gemini Flash.
- Generate narration and dialogue using Gemini Pro.
- Generate panel illustrations using Stable Diffusion.
- Build the comic layout.
- Create the PDF using FPDF.

Output:

- Panel-by-panel comic preview.
- Generated illustrations and narrative content.
- Downloadable multi-page PDF.
- Export success confirmation.

8. User Scenarios

- Scenario 1: A user enters a story such as a brave fox exploring an enchanted forest and receives a personalized comic.
- Scenario 2: A user changes the tone to funny and the art style to comic book, then regenerates the comic with the updated preferences.
- Scenario 3: A user reviews the comic preview and downloads the completed comic as a PDF.

9. Frontend Requirements

- index.html shall collect comic creation inputs.
- comic_preview.html shall display generated comic panels.
- export_success.html shall confirm successful export.
- Jinja2 shall dynamically bind generated content to the templates.
- The interface shall use organized forms, buttons, spacing, rounded panels, and responsive layout practices described in the project.

10. Backend Requirements

- FastAPI shall manage routing and server-side operations.
- routes.py shall coordinate user input and AI-generation functions.
- The backend shall receive form data or JSON requests.
- The backend shall process AI responses and pass them to Jinja2 templates.
- The backend shall coordinate outline generation, story generation, image generation, layout construction, and PDF export.

11. AI Model Requirements

- Gemini Flash: structured 5-panel comic outline generation.
- Gemini Pro: detailed narration and character dialogue.
- Stable Diffusion: comic-style illustration generation.
- The selected combination shall support personalized and visually engaging comic creation.

12. Deployment and Testing Requirements

- Create and activate a Python virtual environment.
- Install project dependencies from requirements.txt or the required package list.
- Configure Gemini and Hugging Face credentials through environment variables.
- Start the application using Uvicorn.
- Verify the homepage, comic generation workflow, API endpoints, image generation, and PDF export locally.
- Use the FastAPI /docs interface for API testing.

13. Requirement Analysis Summary

The requirement analysis establishes the complete set of requirements for ComicCraft. The application must collect user preferences, generate a structured comic story, create corresponding illustrations, display the result through a web interface, and export the final comic as a PDF. The requirements also define the FastAPI backend, Jinja2 frontend, Gemini and Stable Diffusion integration, modular project structure, environment configuration, and local testing workflow.

Source: ComicCraft – AI Comic Story Creator using Gemini Models